

FreeRTOS基础（四）：静态创建任务

于 2024-06-01 13:16:53 发布

上一篇博客，我们讲解了[FreeRTOS](#)中如何动态创建任务，那么这一讲，我们从实战出发，规范我们在FreeRTOS下的编码风格，掌握静态创建任务的编码风格，达到实战应用！

目录

一、空闲任务和空闲任务钩子函数

1.0 为什么会有空闲任务？

1.1 空闲任务 (Idle Task)

1.2 空闲任务钩子函数 (Idle Task Hook)

二、静态创建任务的基本步骤

2.1 使能FreeRTOS的API函数

2.2 定义静态创建任务函数的入口参数

2.3 编写任务函数

2.4 主函数进行调用

2.5 补充

2.6 任务执行顺序

四、静态创建任务和删除任务的API函数解析(选学)

五、静态创建任务和动态创建任务的区别

5.1 创建过程上比较

5.2 整体理解

5.3 应用场合

六、临界区保护

6.1 概念

6.2 临界区保护实现方法

6.3 使用示例

6.4 注意事项

内容来源：csdn.net

作者昵称：未来可期，静待花开~

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139370936

作者主页：https://blog.csdn.net/weixin_47040031

1、实验目的：学会 xTaskCreateStatic() 和 vTaskDelete() 的使用

2、实验设计：将设计四个任务：start_task、task1、task2、task3

四个任务的功能如下：

start_task

用来创建其他的三个任务

task1

实现LED0每500ms闪烁一次

task2

实现LED1每500ms闪烁一次

task3

判断按键KEY0是否按下，按下则删掉task1

CSDN @未来可期，静待花开~

一、空闲任务和空闲任务钩子函数

1.0 为什么会有空闲任务？

思考一个问题:在我们的Free RTOS中可以将所有任务阻塞吗？

不能。

①所有任务都阻塞而没有可运行的任务，会导致系统无法处理外部响应，这与实时操作系统的设计 理念相悖。

②可能会出现死锁或系统崩溃。

因此，必须始终至少有一个任务可以进入运行状态，这就是空闲任务。

1.1 空闲任务 (Idle Task)

在FreeRTOS中，**空闲任务是在我们调用任务调度器函数的内部自动创建的**，它是**优先级最低**的任务。它在系统中没有其他任务可以运行时执行。其主要作用包括：

1. **CPU利用率**：空闲任务的运行表明系统没有其他更高优先级的任务需要运行，因此空闲任务的执行时间可以被用来计算CPU的空闲时间，从而得出CPU利用率。
2. **内存管理**：空闲任务负责清理被删除的任务堆栈内存。

注意：空闲任务不需要我们进行创建！！

注意：空闲任务它也是一个任务，只不过比较特殊而已，它是任务调度器自动帮我们进行创建的，在静态创建任务时，我们首先使能空闲任务的API函数，

cpp

AI写代码 | 复制 | 运行

```
1 // FreeRTOSConfig.h 文件中
2 #define configUSE_IDLE_HOOK 1
```

然后自己实现空闲任务的内存分配函数（因为静态创建的时候，是由我们自己进行分配的，分配在bass段，通常为全局数组），

cpp

AI写代码 | 复制 | 运行

```
1
2 /*** 空闲任务配置 ***/
3 StaticTask_t idle_task_tcb; // 空闲任务控制块
4 StackType_t idle_task_stack[configMINIMAL_STACK_SIZE]; // 空闲任务栈的大小
5
6 /* 空闲任务内存分配函数实现 */
7 void vApplicationGetIdleTaskMemory( StaticTask_t ** ppxIdleTaskTCBBuffer,
8                                     StackType_t ** ppxIdleTaskStackBuffer,
9                                     uint32_t * pulIdleTaskStackSize )
10 {
11     *ppxIdleTaskTCBBuffer=&idle_task_tcb;
12     *ppxIdleTaskStackBuffer=idle_task_stack;
```

内容来源: csdn.net

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139370936

作者主页: https://blog.csdn.net/weixin_47040031

```
13 | * puIdleTaskStackSize=configMINIMAL_STACK_SIZE;14 |  
15 | }
```

如果需要在空闲任务中执行特定操作，可以实现空闲任务钩子函数。

```
cpp AI写代码 | 复制 运行  
  
1 // 空闲任务钩子函数  
2 void vApplicationIdleHook(void)  
3 {  
4     // 在此处添加空闲时要执行的代码  
5 }
```

在动态创建任务时，我们不需要实现空闲任务的内存分配函数，因为这是由 FreeRTOS 自动的从 FreeRTOS 管理的堆中分配。即动态任务的分配不需要管空闲任务。

1.2 空闲任务钩子函数 (Idle Task Hook)

空闲任务钩子函数是一个用户定义的函数，它在每次空闲任务运行时被调用。它允许用户在系统空闲时执行一些特定的任务，比如低功耗模式的处理、监控等。

如何使用空闲任务钩子函数

- step1、配置宏（使能空闲任务API函数）
- step2、实现空闲任务的内存分配函数（注意：只有静态创建任务才需要！）
- step3、自定义实现空闲任务钩子函数（根据自己的需求）

```
cpp AI写代码 | 复制 运行  
  
1 //1、FreeRTOSConfig.h文件中  
2 #define configUSE_IDLE_HOOK 1  
3  
4  
5 ***** 空闲任务配置 *****  
6 StaticTask_t idle_task_tcb; // 空闲任务控制块  
7 StackType_t idle_task_stack[configMINIMAL_STACK_SIZE]; // 空闲任务栈的大小
```

内容来源: csdn.net
作者昵称: 未来可期, 静待花开~
原文链接: https://blog.csdn.net/weixin_47040031/article/details/139870936
作者主页: https://blog.csdn.net/weixin_47040031

```

8
9 | /*2、空闲任务内存分配函数实现*/          只有静态创建任务才需要
10 void vApplicationGetIdleTaskMemory( StaticTask_t ** ppxIdleTaskTCBBuffer,
11                                     StackType_t ** ppxIdleTaskStackBuffer,
12                                     uint32_t * pulIdleTaskStackSize )
13 {
14     *ppxIdleTaskTCBBuffer=&idle_task_tcb;
15     *ppxIdleTaskStackBuffer=idle_task_stack;
16     * pulIdleTaskStackSize=configMINIMAL_STACK_SIZE;
17
18 }
19
20
21
22 //3、空闲任务钩子函数具体实现（根据需求）
23 void vApplicationIdleHook(void)
24 {
25     // 在此处添加空闲时要执行的代码
26 }

```

通过上述配置，空闲任务钩子函数将在系统空闲时被调用，执行用户定义的代码。

二、静态创建任务的基本步骤

1、需将宏configSUPPORT_STATIC_ALLOCATION 配置为 1

2、定义空闲任务(必须要)&定时器任务（可选）的任务堆栈及TCB

3、实现两个接口函数

4、定义函数入口参数

5、编写任务函数

vApplicationGetIdleTaskMemory() 传参用的，**必须有**

vApplicationGetTimerTaskMemory() 可选

用起来只需这五步

内容来源：csdn.net

作者昵称：未来可期，静待花开~

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139370936

作者主页：https://blog.csdn.net/weixin_47040031

2.1 使能FreeRTOS的API函数

同样在使用FreeRTOS任务创建函数之前，我们需要在配置文件里（FreertosConfig.h）使能API函数

1. 需将宏configSUPPORT_STATIC_ALLOCATION 配置为 1，此时便支持静态创建。利用Ctrl+F搜索即可。
2. 使能空闲任务的API函数，在FreeRTOSConfig.h文件中**#define configUSE_IDLE_HOOK 1**

```
85 #define configSUPPORT_STATIC_ALLOCATION 1 //静态创建任务
86 // #define configSUPPORT_DYNAMIC_ALLOCATION 1 //支持动态内存申请

42 *空闲钩子函数的使用与否
43 *****/
44 #define configUSE_IDLE_HOOK 1
```

CSDN @未来可期，静待花开~

2.2 定义静态创建任务函数的入口参数

通过前面的讲解，我们知道动态创建任务的API函数如下：

TaskHandle_t xTaskCreateStatic

```
(
    TaskFunction_t 函数指针    pxTaskCode,
    const char * const 字符串    pcName,
    const uint32_t 长整型    ulStackDepth,
    void * const void*    pvParameters,
    UBaseType_t 无符号长整型    uxPriority,
    StackType_t * const    puxStackBuffer,
    StaticTask_t * const    pxTaskBuffer
);
```

/* 指向任务函数的指针 */
/* 任务函数名 */
/* 任务堆栈大小**注意字为单位** */
/* 传递的任务函数参数 */
/* 任务优先级 */ **定义全局数组即可，传入数组名**
/* 任务堆栈，**一般为数组，由用户分配** */
/* 任务控制块指针，**由用户分配** */

返回值为任务的句柄，也就是任务控制块指针

CSDN @未来可期，静待花开~
CSDN @未来可期，静待花开~

与动态创建的任务相比，只是后面两个参数发生了变化，其实这在前面讲过了，这是因为：**静态创建任务时，任务的任务控制块以及任务的栈空间所需的内存，需用户分配提供，通常在bass段申请一个足够大的全局数组即可！该函数返回该任务的任务句柄！**

其实，我们需要定义的入口参数就是这个API函数的参数，提前定义好，然后传入参数，他就会自动的为我们创建好对应的任务，并且处于一种就绪态。从上面我们可以看到：

1、任务函数指针：

其实就是函数名，我们知道函数名就是函数的入口地址，就是一个函数指针

2、任务名字：

其实也就是函数名对应的字符串，要用双引号括起来

3、任务堆栈大小：

静态创建任务，**任务的任务控制块以及任务的栈空间所需的内存，均需要我们自己进行内存分配，通常在bass段申请一个足够大的全局数组即可！我们需要定义好任务栈的大小，也就是数组的大小！**使用宏：

cpp

AI写代码 | 复制 | 运行

```
#define START_TASK_STACK_SIZE 128 // 定义堆栈大小为128字（1字等于4字节）
```

4、传递给任务的参数：

不需要传参，我们直接给NULL即可；

内容来源：csdn.net

作者昵称：未来可期，静待花开~

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139970936

作者主页：https://blog.csdn.net/weixin_47040031

5、任务优先级:

我们使用的是硬件的方式, 因此, 它要在0-31之间, 使用宏定义即可:

cppAI写代码 | 复制 | 运行

```
#define START_TASK_PRIORITY 1 //定义任务优先级, 0-31 根据任务需求
```

6、任务堆栈地址:

这个参数就是我们自己申请的任务堆栈的地址, 我们使用的是全局数组, 使用宏定义即可, 需要我们提前定义好, 然后传入数组名即可, 因为数组名就是地址。

cppAI写代码 | 复制 | 运行

```
StackType_t start_task_stack[START_TASK_STACK_SIZE]; //申请的任务堆栈(全局数组)
```

7、任务控制块地址:

动态创建任务, **任务控制块也是由我们自己进行内存分配的, 任务控制块就是一个结构体, 因此需要我们提前定义好任务控制块, 然后传入任务控制块的地址。使用宏即可:**

cppAI写代码 | 复制 | 运行

```
StaticTask_t start_task_tcb; //创建任务控制块结构体
```

8、任务句柄:

这个是因为: 这个**函数的返回值为任务句柄**, 我们后续对任务的删除等操作, 都是通过该任务句柄进行操作, 因此, **我们需要提前定义好任务句柄, 然后接收创建任务的返回值即可对该任务进行操作!** 使用宏即可:

cppAI写代码 | 复制 | 运行

```
TaskHandle_t start_task_handler; //定义任务句柄(结构体指针)
```

从上面我们可以知道: 其实我们只需要提前利用宏定义好五个参数即可, 其他的参数只要任务函数编写好, 便可以确定。示例如下:

cpp内容来源: csdn.net AI写代码 | 复制 | 运行

```
1 //*****START_TASK任务配置*****  
2 //*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****
```

作者昵称: 未来可期, 静待花开~
原文链接: https://blog.csdn.net/weixin_47040031/article/details/139870936
作者主页: https://blog.csdn.net/weixin_47040031

```

3 | 4 | #define      START_TASK_STACK_SIZE  128    //定义堆栈大小为128字(1字等于4字节)
5 | #define      START_TASK_PRIO          1      //定义任务优先级, 0-31 根据任务需求
6 | StackType_t   start_task_stack[START_TASK_STACK_SIZE]; //申请的任务堆栈(全局数组)
7 | StaticTask_t  start_task_tcb;              //申请的任务控制块(创建结构体)
8 | TaskHandle_t  start_task_handler;          //定义任务句柄(结构体指针)
9 | void start_task(void* args);

```

注意:

1. 为了编码规范, 我们使用的宏都是大写, 虽然较长, 但是通俗易懂;
2. 使用API函数进行任务创建, 里面的参数需要进行强制转换, 以免报错。
3. 为了任务执行的顺序是按照我们设定好的优先级执行的, 我们可以在创建任务的任务中, 使用临界段保护, 那么在这个任务体中, 可以屏蔽中断(中断优先级在5-15之内) 比如切换任务的PendSV, 此时, 我们创建任务的过程中, 不会进行任务的调度, 然后我们创建任务结束后, 在打开临界段保护, 此时不会对所有中断进行屏蔽, 也就是任务切换PendSV(中断)才会进行任务调度。如下代码所示, 在创建任务开始之前和创建任务之后加入, 后面详细讲解。
4. 静态创建任务函数, 有返回值, 返回任务句柄, 用提前定义好的任务句柄接收, 后面便可以使用任务句柄操作任务。

cpp

AI写代码 | 复制 | 运行

```

1 | #include "stm32f4xx.h"           // Device header
2 | #include "stdio.h"
3 | #include "FreeRTOS.h"
4 | #include "task.h"
5 | #include "dynamic.h"
6 |
7 |
8 |
9 | /*****START_TASK任务配置*****/
10 | /*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
11 |
12 | #define      START_TASK_STACK_SIZE  128    //定义堆栈大小为128字(1字等于4字节)
13 | #define      START_TASK_PRIO          1      //定义任务优先级, 0-31 根据任务需求
14 | TaskHandle_t start_task_handler;          //定义任务句柄(结构体指针)
15 | StackType_t  start_task_stack[START_TASK_STACK_SIZE];

```

内容来源: csdn.net

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139870936

作者主页: https://blog.csdn.net/weixin_47040031

```

16 StaticTask_t start_task_tcb; 17 | void start_task(void* args);
18
19
20
21
22 /*****TASK1任务配置*****/
23 /*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
24 #define TASK1_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
25 #define TASK1_PRIIO 2 //定义任务优先级，0-31根据任务需求
26 TaskHandle_t task1_handler; //定义任务句柄（结构体指针）
27 StackType_t task1_stack[TASK1_STACK_SIZE];
28 StaticTask_t task1_tcb;
29 void task1(void* args);
30
31
32
33
34 /*****TASK2任务配置*****/
35 /*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
36 #define TASK2_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
37 #define TASK2_PRIIO 3 //定义任务优先级，0-31根据任务需求
38 TaskHandle_t task2_handler; //定义任务句柄（结构体指针）
39 StackType_t task2_stack[TASK2_STACK_SIZE];
40 StaticTask_t task2_tcb;
41 void task2(void* args);
42
43
44
45 /*****TASK3任务配置*****/
46 /*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
47 #define TASK3_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
48 #define TASK3_PRIIO 4 //定义任务优先级，0-31根据任务需求
49 TaskHandle_t task3_handler; //定义任务句柄（结构体指针）
50 StackType_t task3_stack[TASK3_STACK_SIZE];
51 StaticTask_t task3_tcb;
52 void task3(void* args);
53
54
55

```

内容来源: csdn.net

作者昵称: 未来可期，静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139670936

作者主页: https://blog.csdn.net/weixin_47040031

```
1  /******开始任务用来创建其他三个任务，只创建一次，不能是死循环，同时创建完3个任务后删除任务1本身******/
2  void start_task(void* args)
3  {
4      taskENTER_CRITICAL();    /*进入临界区*/
5
6      task1_handler = xTaskCreateStatic( (TaskFunction_t)      task1,
7                                         ( char * )          "task1",
8                                         (uint32_t)           TASK1_STACK_SIZE,
9                                         (void * )           NULL,
10                                        (UBaseType_t)          TASK1_PRI0,
11                                        (StackType_t * )        task1_stack,
12                                        (StaticTask_t * )        &task1_tcb );
13
14      task2_handler= xTaskCreateStatic( (TaskFunction_t)      task2,
15                                       ( char * )          "task2",
16                                       (uint32_t)           TASK2_STACK_SIZE,
17                                       (void * )           NULL,
18                                       (UBaseType_t)          TASK2_PRI0,
19                                       (StackType_t * )        task2_stack,
20                                       (StaticTask_t * )        &task2_tcb );
21
22      task3_handler= xTaskCreateStatic( (TaskFunction_t)      task3,
23                                       ( char * )          "task3",
24                                       (uint32_t)           TASK3_STACK_SIZE,
25                                       (void * )           NULL,
26                                       (UBaseType_t)          TASK3_PRI0,
27                                       (StackType_t * )        task3_stack,
28                                       (StaticTask_t * )        &task3_tcb );
29
30
31
32      vTaskDelete(start_task_handler);    // 删除开始任务自身，传参NULL或者开始任务句柄
33
34      taskEXIT_CRITICAL();    /*退出临界区*/
35
```

内容来源: [csdn.net](https://blog.csdn.net/weixin_47040031)

作者昵称: 未来可期，静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139670936

作者主页: https://blog.csdn.net/weixin_47040031

```

36 | 37 | //临界区内不会进行任务的调度切换，出了临界区才会进行任务调度，抢占式
38 | }

```

此外，还要**实现空闲任务的内存分配函数**，至于空闲任务的钩子函数可根据自己需求是否实现，这里就不实现了。

```

cpp AI写代码 | 复制 运行

1  /*****空闲任务配置*****/
2  StaticTask_t idle_task_tcb;           //空闲任务控制块
3  StackType_t  idle_task_stack[configMINIMAL_STACK_SIZE]; //空闲任务栈的大小
4
5  /*空闲任务内存分配*/
6  void vApplicationGetIdleTaskMemory( StaticTask_t ** ppxIdleTaskTCBBuffer,
7                                     StackType_t ** ppxIdleTaskStackBuffer,
8                                     uint32_t * pulIdleTaskStackSize )
9  {
10     *ppxIdleTaskTCBBuffer=&idle_task_tcb;
11     *ppxIdleTaskStackBuffer=idle_task_stack;
12     * pulIdleTaskStackSize=configMINIMAL_STACK_SIZE;
13
14 }

```

2.3 编写任务函数

对每个任务具体实现的功能进行函数的实现：**需要注意，任务函数没有返回值并且是死循环的！**

```

cpp AI写代码 | 复制 运行

1  /*****其余三个任务的任务函数，无返回值且是死循环*****/
2
3  /***任务1：实现LED0每500ms翻转一次*****/
4  void task1(void* args)
5  {
6      while(1)
7      {
8          printf("任务1正在运行！\n");

```

内容来源：csdn.net
 作者昵称：未来可期，静待花开~
 原文链接：https://blog.csdn.net/weixin_47040031/article/details/139870936
 作者主页：https://blog.csdn.net/weixin_47040031

```

9      GPIO_ToggleBits(GPIOF,GPIO_Pin_9 );
      10      vTaskDelay(500);          //FreeRTOS 自带的延时函数
11
12  }
13
14
15  }
16
17  /** 任务2：实现LED1 每500ms 翻转一次***/
18  void task2(void* args)
19  {
20      while(1)
21      {
22          printf("任务2正在运行! \n");
23          GPIO_ToggleBits(GPIOF,GPIO_Pin_10 );
24          vTaskDelay(500);          //FreeRTOS 自带的延时函数
25      }
26
27
28  }
29
30
31  /** 任务3：判断按键KEY0，按下KEY0, 任务1删除***/
32  void task3(void* args)
33  {
34      while(1)
35      {
36          printf("任务3正在运行! \n");
37          if(GPIO_ReadInputDataBit(GPIOE,GPIO_Pin_4)==0) //表示按键按下
38          {
39              if(task1_handler!=NULL) //防止重复删除
40              {
41                  printf("删除任务1! \n");
42                  vTaskDelete(task1_handler); //删除任务1，传任务1的句柄
43                  task1_handler=NULL;
44              }
45          }
46          vTaskDelay(10);
47      }

```

内容来源: csdn.net

作者昵称: 未来可期，静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139870936

作者主页: https://blog.csdn.net/weixin_47040031

```
48 |  
49 |  
50 | }
```

此外，我们再自定义一个入口函数，用来创建开始任务，然后将要创建的任务全部放在这个开始任务中，主函数只需调用这个入口函数，即可在这个开始任务中，创建其他的任务，这样做，规范代码，梳理代码逻辑，清晰易懂任务的运行顺序！如下所示：

cpp

AI写代码 | 复制 | 运行

```
1 //FreeRTOS入口例程函数，无参数，无返回值，用来创建开始任务  
2 void freertos_demo(void)  
3 {  
4     //静态创建任务会返回任务句柄  
5  
6     start_task_handler = xTaskCreateStatic( (TaskFunction_t) start_task,  
7                                             (char *) "start_task",  
8                                             (uint32_t) START_TASK_STACK_SIZE,  
9                                             (void *) NULL,  
10                                            (UBaseType_t) START_TASK_PRIO,  
11                                            (StackType_t *) start_task_stack,  
12                                            (StaticTask_t *) &start_task_tcb );  
13  
14  
15     vTaskStartScheduler(); //开启任务调度器  
16  
17 }
```

2.4主函数进行调用

在完成上述的编写后，主函数内部只需要引入对应的头文件，然后在函数内部调用相应的函数对使用到的外设进行初始化，然后调用入口函数即可进行按照我们设定的优先级进行任务的调度，如下所示：

cpp

AI写代码 | 复制 | 运行

```
1 #include "stm32f4xx.h" // Device header  
2 #include "stdio.h"  
3 #include "myled.h"  
4 #include "mykey.h"
```

内容来源: [csdn.net](https://blog.csdn.net/weixin_47040031)
作者昵称: 未来可期，静待花开~
原文链接: https://blog.csdn.net/weixin_47040031/article/details/139870936
作者主页: https://blog.csdn.net/weixin_47040031

```

5 | #include "myusart.h" 6 |
7 | #include "FreeRTOS.h"
8 | #include "task.h"
9 | #include "dynamic.h" //可以用来单独存放任务函数的声明以及配置相关的宏定义，然后直接引入头文件使用
10 |
11 | extern TaskHandle_t Start_Handle;
12 | /*使用任务句柄可以对任务操作，如果没有添加上面的单独头文件存放，
13 | 那么使用其他文件的全局变量利用extern关键字引入即可。*/
14 |
15 | int main(void)
16 | {
17 |     //1、外设初始化
18 |     My_UartInit();
19 |     LED_Init();
20 |     KEY_Init();
21 |
22 |
23 |     //2、调用入口函数
24 |     freertos_demo();
25 |
26 |
27 |
28 | }

```

2.5补充

为进行模块化的编程，我们可以将创建相应的头文件可以用来单独存放任务函数的声明以及任务配置相关的宏定义，然后在主函数直接引入头文件使用即可，这样工程结构清晰易懂！

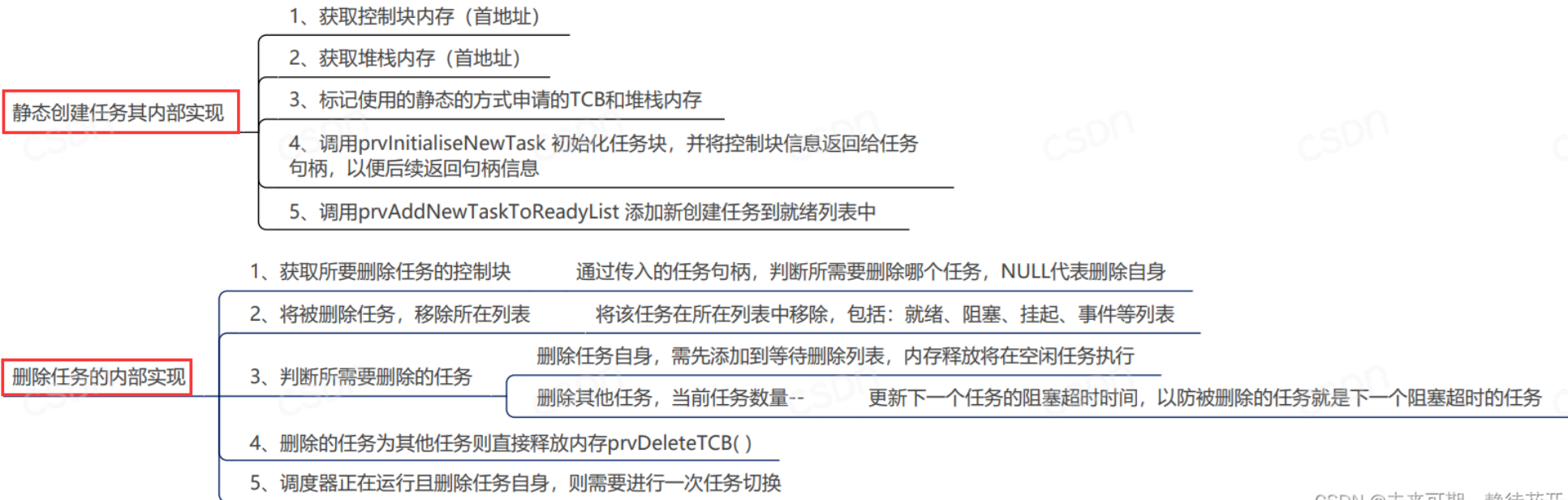
2.6任务执行顺序

编写完程序后，一定要进行验证，验证程序是否按照我们设定的顺序及进行执行，类似于[操作系统](#)的线程同步问题！

首先主函数调用入口函数，在入口函数内部创建开始任务函数，该开始任务进入就绪状态，启用任务调度器，调度器启动后，FreeRTOS 将接管系统控制，开始调度任务。此时CPU就会去执行开始任务，然后，在开始任务中创建三个任务，注意：由于使用了临界保护：taskENTER_CRITICAL(); /*进入临界区*/ 它会对5-15优先级的中断进行屏蔽，即不会发生作用，其中PendSV是用来任务切换的内核中断，它的优先级是13，因此，会被屏蔽，也就是说，我在创建三个任务的过程中，不会进行其他任务的切换，保证我的开始任务创建其他的三个任务不会被打断！！！创建完三个任务后，它们都进入了就

绪态，然后，再删除这个开始任务（因为每个任务只需要创建一次，多次创建占用堆栈内存，造成栈溢出！）此时，我在关闭临界区保护，
taskEXIT_CRITICAL(); /*退出临界区*/，也就是打开所有中断，此时PendSV中断就会被打开，按照任务的优先级进行抢占式调度，分别执行任务3、任务2、任务1，在三个任务执行的过程中，加入适当的延时，他就会进行任务的切换，去就绪列表寻找优先级最高的任务去运行！

四、静态创建任务和删除任务的API函数解析(选学)



CSDN @未来可期，静待花开~

五、静态创建任务和动态创建任务的区别

5.1 创建过程上比较

动态创建的任务，任务的任务控制块以及任务的栈空间所需的内存，均由 FreeRTOS 从 FreeRTOS 管理的堆中分配（真正需要我们定义的也就是任务句柄）；

内容来源：csdn.net
作者昵称：未来可期，静待花开~
原文链接：https://blog.csdn.net/weixin_47040031/article/details/139970936
作者主页：https://blog.csdn.net/weixin_47040031

```

1  /*****START_TASK任务配置*****/
2  /*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
3  #define START_TASK_STACK_SIZE 128 // 定义堆栈大小为128字 (1字等于4字节)
4  #define START_TASK_PRIO 1 // 定义任务优先级, 0-31根据任务需求
5  TaskHandle_t start_task_handler; // 定义任务句柄 (结构体指针)
6  void start_task(void* args);

```

CSDN @未来可期, 静待花开~

静态创建的任务, 任务的任务控制块以及任务的栈空间所需的内存, 需用户分配提供。
需要我们额外定义这两个。

```

/*****START_TASK任务配置*****/
/*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/

#define START_TASK_STACK_SIZE 128 // 定义堆栈大小为128字 (1字等于4字节)
#define START_TASK_PRIO 1 // 定义任务优先级, 0-31根据任务需求
StackType_t start_task_stack[START_TASK_STACK_SIZE]; // 申请的任务堆栈 (全局数组)
StaticTask_t start_task_tcb; // 申请的任务控制块 (创建结构体)
TaskHandle_t start_task_handler; // 定义任务句柄 (结构体指针)
void start_task(void* args);

```

CSDN @未来可期, 静待花开~

此外, 静态创建任务, 需要我们实现空闲任务的内存分配函数。

内容来源: csdn.net

作者昵称: 未来可期, 静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139370936

作者主页: https://blog.csdn.net/weixin_47040031

```

/*****空闲任务配置*****/
StaticTask_t idle_task_tcb; // 空闲任务控制块
StackType_t idle_task_stack[configMINIMAL_STACK_SIZE]; // 空闲任务栈的大小

/* 空闲任务内存分配 */
void vApplicationGetIdleTaskMemory( StaticTask_t ** ppxIdleTaskTCBBuffer,
                                     StackType_t ** ppxIdleTaskStackBuffer,
                                     uint32_t * pulIdleTaskStackSize )
{
    *ppxIdleTaskTCBBuffer=&idle_task_tcb;
    *ppxIdleTaskStackBuffer=idle_task_stack;
    * pulIdleTaskStackSize=configMINIMAL_STACK_SIZE;
}

```

CSDN @未来可期，静待花开~

5.2 整体理解

1. **静态创建任务是在编译时为任务分配内存（全局数组）**，任务在运行之前已分配内存。这种方法不需要在运行时使用动态内存分配函数，因此更加可靠和节省内存。
2. 动态创建任务是在运行时通过动态内存分配函数分配任务内存。这种分配方式可以更灵活地适应不同大小和数量的任务。然而，在动态方式下，程序需要在运行时使用动态内存分配函数，这可能会导致内存泄漏和堆碎片等问题。

特点	静态创建任务	动态创建任务
内存分配	编译时预分配	运行时动态分配
内存管理确定性	高 (没有内存碎片)	低 (可能会有内存碎片)
灵活性	较低 (需要预先知道任务数量和堆栈大小)	高 (可以根据需要在运行时创建和删除任务)
适用场景	高可靠性系统, 资源受限的嵌入式系统	需要灵活任务管理的系统
内存使用效率	预先分配可能导致未使用的内存浪费	更有效的内存使用, 但需注意碎片化问题
任务创建失败处理	编译时错误, 容易检测	运行时错误, 需处理内存不足等情况

CSDN @未来可期，静待花开~

5.3 应用场合

- 1、在实际的应用中，动态方式创建任务是比较常用的，除非有特殊的需求，一般都会使用动态方式创建任务，动态创建相对简单，更为常用，静态创建:可将任务堆栈放置在特定的内存位置，并且无需关心对内存分配失败的处理。
- 2、如果应用程序中的任务数量和大小已知，则可以使用静态方式分配内存，并且无需动态内存分配。如果应用程序需要更多的灵活性，并且需要在运行时根据需要创建或删除任务，则应使用动态方式分配内存。

六、临界区保护

6.1 概念

在FreeRTOS（实时操作系统）中，临界区保护是一个关键概念，用于确保共享资源在多任务环境中被安全地访问和修改。临界区保护防止多个任务在同一时间访问共享资源，从而避免数据竞争和不一致的情况。它可以保护那些不想被打断的程序段,关闭freertos所管理的中断，中断无法打断，滴答中断和PendSV中断无法进行不能实现任务调度。

6.2 临界区保护实现方法

内容来源: csdn.net

作者昵称: 未来可期，静待花开~

原文链接: https://blog.csdn.net/weixin_47040031/article/details/139370936

作者主页: https://blog.csdn.net/weixin_47040031

在FreeRTOS中，临界区保护主要通过以下两种方法实现：

1. 任务级临界区保护：

1. **vTaskSuspendAll() 和 xTaskResumeAll()：**这对函数用于挂起和恢复任务调度。当调用vTaskSuspendAll()时，FreeRTOS内核暂停任务切换，直到调用xTaskResumeAll()。在此期间，任务调度被禁止，确保当前任务独占地执行临界区内的代码。
2. 适用于临界区执行时间较长的场景，因为任务调度被完全禁止。

2. 中断级临界区保护：（常用方法）

1. **taskENTER_CRITICAL() 和 taskEXIT_CRITICAL()：**这对宏用于进入和退出临界区。当调用taskENTER_CRITICAL()时，内核禁用所有中断，确保在临界区内的代码不会被任何中断打断。当调用taskEXIT_CRITICAL()时，恢复中断的状态。
2. 适用于临界区执行时间较短的场景，因为禁用中断的时间较短，能减少系统的中断响应延迟。

6.3 使用示例

任务级临界区保护示例：

```
cpp AI写代码 | 复制 运行

1 void vTaskFunction(void *pvParameters)
2 {
3     // 其他代码
4     vTaskSuspendAll(); // 挂起任务调度
5
6     // 临界区代码
7     // 访问或修改共享资源
8     // 例如：共享变量++
9     sharedVariable++;
10
11     xTaskResumeAll(); // 恢复任务调度
12     // 其他代码
13 }
```

中断级临界区保护示例：

内容来源：csdn.net

作者昵称：未来可期，静待花开~

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139370936

作者主页：https://blog.csdn.net/weixin_47040031

cpp

```
1 void vTaskFunction(void *pvParameters)
2 {
3     // 其他代码
4     taskENTER_CRITICAL(); // 进入临界区，禁用中断
5
6     // 临界区代码
7     // 访问或修改共享资源
8     // 例如：共享变量++
9     sharedVariable++;
10
11    taskEXIT_CRITICAL(); // 退出临界区，恢复中断
12    // 其他代码
13 }
```

6.4 注意事项

临界区保护的注意事项

1. **使用范围**：临界区保护应该仅用于保护需要保护的代码，避免长时间禁用任务调度或中断，影响系统的实时性能。
2. **开销**：禁用任务调度和中断都有一定的开销，特别是禁用中断会影响系统的实时响应，需要慎重使用。
3. **嵌套**：在FreeRTOS中，taskENTER_CRITICAL()和taskEXIT_CRITICAL()可以嵌套使用，但要确保成对出现，防止中断长时间被禁用。

通过以上的介绍，相信你对静态创建任务的过程会有清晰的认识，其实步骤也是非常简单的，接下来去实践吧！熟练后就不难了，万事开头难！至此，静态创建任务就已经讲解完毕！初次学习，循序渐进，一步步掌握即可！以上就是全部内容！请务必掌握，创作不易，欢迎大家点赞加关注评论，您的支持是我前进最大的动力！下期再见！

标签 [#java](#) [#开发语言](#)

内容来源：csdn.net

作者昵称：未来可期，静待花开~

原文链接：https://blog.csdn.net/weixin_47040031/article/details/139370936

作者主页：https://blog.csdn.net/weixin_47040031